업무일지 – 2026-10-27(화) · Day 1

오늘 할 일 (시작 전)

- [x] 개인 프로젝트 안내와 초급 프로젝트 가이드 숙지
- [x] 제공 데이터의 폴더 구조, 이미지·라벨 형식 파악
- [x] 프로젝트 일정 수립 (마감 역산)

한 일

- 가이드를 읽고 과제를 정리했다: 한 이미지에서 최대 4개 알약의 클래스와 바운딩 박스를 검출하는 객체 탐지 문제다. 평가는 Kaggle 비공개 대회에서 한다.
- 사용 금지 데이터를 확인했다: AI Hub 경구약제조합 5000종의 TL_2_조합.zip, TS_2_조합.zip
- 데이터 구조 파악 (Python 스크립트로 집계)
  - train_images 232장, test_images 842장, 라벨 JSON 763개. 해상도는 모두 976×1280이다.
  - 라벨은 COCO 형식이지만 **JSON 하나에 알약 하나**만 있고, 조합 폴더 / 약 코드 폴더로 흩어져 있다. 이미지별로 합쳐야 한다.
  - 클래스 56종. `category_id`는 약 코드 숫자라서 YOLO용으로 0~55로 다시 매겨야 한다.
  - 이미지당 알약 수: 3개 157장, 4개 75장
  - 촬영 조건: 배경·조명·크기가 모두 같다. 카메라 각도만 70/75/90 세 가지다. 앞면이 92%다.

발견한 문제 (인사이트)

1. **데이터가 적다**: 학습 232장으로 테스트 842장을 맞혀야 한다. 사전학습 모델 파인튜닝과 증강이 필수다.

2. **클래스 불균형**: 최다 153개(일양하이트린정 2mg)부터 최소 3개까지

3. **라벨 누락**: 파일명상 알약 수보다 박스가 적은 이미지가 8장 있다. 그대로 쓰면 빠진 알약이 배경으로 학습된다.

4. **잘못된 박스**: `K-003351-016262-018357_0_2_0_2_75_000_200.png`의 x=6567로 이미지 폭(976)을 넘는다.

추가로 한 일 (오후)

- 레포 폴더 구조(`pill-detection/`), README, .gitignore, requirements.txt 작성
- EDA 노트북 `notebooks/01_eda.ipynb` 작성, 로컬에서 실제 데이터로 끝까지 실행해 검증 (그래프 14장, class_map.csv, image_meta.csv)
- 새로 발견한 것
  - 라벨 누락 8장 모두 **빠진 알약이 사진에 실제로 있다** → 박스 보완이 필요하다
  - 잘못된 박스 x=6567은 **x=567의 입력 오류**다 (후보 좌표를 그려 확인)
  - 17개 클래스는 같은 장면을 각도만 바꾼 3장뿐이다 → Copy-Paste 증강 검토
  - 같은 조합을 각도만 바꿔 찍었다 → 이미지 단위 분할은 누수. **조합 단위 분할**로 검증한다
  - 박스가 고정 슬롯 위치에 몰려 있다. Test는 Train보다 약간 어둡고 대비가 크다

추가로 한 일 (저녁) – 전처리 노트북 `02_preprocess.ipynb`

- 라벨을 검증할 **독립적인 기준**으로, 배경 차분(Lab 색 거리 + Otsu)으로 알약 영역을 분할했다.
- 라벨 박스와 분할 영역을 대조해 라벨을 감사했고, 오류 유형별 규칙으로 자동 수정했다 → **10건 수정**

(범위 밖 1, 엉뚱한 위치 1, 누락 8). 전/후 그림으로 전부 눈으로 검수했다.

- **실수와 교훈**: 처음 만든 "빈 박스 이동" 규칙이 6건을 고쳤는데 5건이 오탐이었다.

반투명 캡슐(OMACOR)과 파란 알약은 분할이 잘 안 돼서 "박스 안이 비었다"고 잘못 판단한 것이다.

→ "원래 박스와 겹치지 않고 크기가 비슷한 알약일 것"이라는 조건을 더해 1건(진짜 오류)만 남겼다.

자동 규칙은 반드시 결과를 눈으로 확인해야 한다는 걸 배웠다.

- 박스 보정: 분할 박스와 라벨러 박스의 테두리 차이(중앙값)로 새 박스를 라벨러 기준에 맞췄다.
- 조합 단위 Stratified Group 5-Fold로 나누고 누수가 0인지 `assert`로 확인했다.

fold마다 val에 train에서 못 본 클래스가 1~7개 있다 (1개 조합에만 있는 클래스 때문).

- YOLO 라벨을 다시 읽어 픽셀로 되돌려 비교했다 (최대 오차 0.001px).
- 기술 메모: Windows에서 `cv2.imread`는 한글 경로를 못 읽어서 PIL로 읽었다.

추가로 한 일 (밤) – 저장소 정리와 계획 재수립

- 프로젝트 코드를 데이터 폴더에서 분리해 `ai-project-01-pill-detection` 저장소로 GitHub에 올렸다.

스프린트 프로젝트가 3개라서 이름을 `ai-project-0N-주제` 패턴으로 통일했다.

- README를 포트폴리오용으로 다시 구성했다 (핵심 성과, 파이프라인 다이어그램, 그래프, 검증 전략).
- **계획을 다시 세웠다.** 처음 일정표는 단순했고, 가이드를 다시 대조해 보니 빠진 항목이 많았다

(모델 비교, 하이퍼파라미터 튜닝, 증강, 파이프라인 자동화, 시연 영상 등).

가이드의 모든 항목을 요구사항 추적표(R1–R17)로 만들고, 산출물·완료 기준·날짜를 붙였다.

- 핵심 설계 결정: **2단계 구조 (YOLO 검출 + CNN 분류기)**
  - 근거: 56종에 비해 박스가 771개뿐이고 29종은 10개 미만이다. 검출은 사실상 "알약이냐 아니냐" 문제라 데이터가 충분하다. 종류 판별은 crop 분류기에 강한 증강을 주는 편이 유리하다.
  - 가이드의 VGG / ResNet / YOLO 비교도 이 구조 안에서 자연스럽게 수행된다.
  - 단일 YOLO 베이스라인과 같은 fold에서 비교해 실제로 나은지 확인한다.
- 도메인 지식 활용 아이디어: 이미지당 알약 최대 4개(`max_det=4`), 같은 약 중복 없음

추가로 한 일 – 코드 구조화 (파이프라인 자동화)

- 문제: 코드가 노트북 셀 안에만 있어서, 학습·추론 노트북에서 같은 코드를 다시 복사해야 하고 리뷰어가 코드를 찾기 어려웠다.
- 해결: 공통 로직을 `src/pilldet` 패키지 8개 모듈로 분리했다 (config, data, boxes, segment, clean, split, export, viz).
  - `scripts/prepare_data.py`: 데이터 준비 전체를 명령 한 줄로 실행 (8.7초), 로그는 `reports/logs/`에 저장

    - `tests/test_core.py`: IoU, 좌표 변환 왕복, 파일명 파싱, fold 누수 방지 등 단위 테스트 11개 통과
    - 노트북은 패키지를 불러 쓰고, 핵심 함수의 실제 코드를 `inspect`로 노트북 안에 함께 보여 준다
- **검증**: 리팩터링 전후의 산출물(label_fixes, folds, class_map, COCO)을 `git diff`로 비교 → **완전히 동일**
- Kaggle에서는 노트북이 GitHub 레포를 clone 해서 같은 패키지를 쓴다

추가로 한 일 – 베이스라인(E0)과 Copy-Paste 증강

- 학습 환경: 로컬 RTX 3060 (12GB)에서 학습한다. Kaggle GPU 주간 한도를 아낄 수 있다.
- 학습·예측·평가·제출 모듈(`train`, `predict`, `evaluate`, `submission`)과 실험 실행 스크립트를 만들었다.

설정 파일 하나가 실험 하나이고, 명령 한 줄로 학습 → 평가 → 제출 파일 → 실험 기록까지 실행된다. 테스트 14개 통과.

- 1에포크 시험 실행에서 문제 2개를 먼저 잡았다: Windows의 OpenMP 런타임 충돌, 크기 0 박스가 제출 파일에 섞이는 문제.
- **E0 결과** (YOLO11s, 640px, fold 0, 5.7분): val mAP50-95 0.835
    - Ultralytics 평가 0.834와 직접 구현한 COCO 평가 0.835가 일치 → 평가 코드 검증
    - **핵심 발견**: 학습에 있는 약은 AP 0.985, 처음 보는 약 5종은 0.000. 점수 손실의 전부가 "처음 보는 약"이다.

위치는 모두 맞고, 처음 보는 약을 생김새가 비슷한 아는 약으로 높은 확신(0.9 이상)으로 예측한다.

    - 최종 모델은 전체 데이터로 학습하므로, 실험 비교 지표에 "학습한 약만 mAP"를 추가했다.
    - 가설 "이미지당 상위 4개 제한이 좋을 것" → **기각** (0.835 → 0.822). mAP는 낮은 순위 후보도 점수로 쳐 준다.
- **Copy-Paste 합성 구현**: 알약을 윤곽대로 오려 다른 사진의 빈 자리에 붙인다. 희귀할수록 자주 뽑는다.
    - 문제: 흰 알약은 배경과 색이 비슷해 절반만 오려지는데 라벨은 전체 박스 → 잘못된 라벨이 된다.
    - 해결: "알약은 라벨 박스를 거의 꽉 채운다"는 점을 이용해, 마스크가 박스의 80% 미만만 덮으면 타원 마스크로 대체. 볼록 껍질로 각인 구멍도 채웠다.
    - fold 0 합성 400장 → 클래스별 박스 수 중앙값 9 → 20
- 실험 대기열: E1(해상도 960·1280), E2(YOLO11n·11m·26s), E3(상하 뒤집기·색 증강·Copy-Paste)

추가로 한 일 – 실험 E1~E3, 분류기·2단계·앙상블

- **E1~E3 (fold 0)**: 해상도(960·1280), 모델 크기(11n·11m·YOLO26s), 증강(상하 뒤집기·밝기·Copy-Paste)을 E0에서 한 가지씩만 바꿔 비교.
    - YOLO11m 0.839가 최고였지만 E0(0.835)와 차이가 작다. 해상도 확대·YOLO26s·Copy-Paste 400장은 오히려 하락.
    - fold 0 검증에는 학습에 없는 약이 5종 있어 mAP50 상한이 28/33 = 0.848이다. 상위 후보들이 0.847로 거의 닿아, **fold 하나로는 더 이상 구분이 안 된다**고 판단했다.
- **분류기 3종 5-Fold 비교** (VGG16 · ResNet50 · EfficientNet-B0): F1 0.989 / 0.987 / **0.994**. 파라미터가 1/33인 EfficientNet-B0을 선택.
- **2단계 모델**(YOLO 박스 + 분류기): 0.832~0.834로 YOLO 단독과 같거나 낮음 → 음성 결과로 기록.

- **WBF 앙상블 직접 구현**: 원 논문 저자 구현과 같은 규칙으로 작성하고 테스트를 붙였다. fold 0에서 5개 모델 WBF 0.841.

추가로 한 일 – 코드 리뷰 반영

- 외부 리뷰에서 지적된 문제를 모두 코드로 확인하고 고쳤다.
  - 5개 fold가 다 있을 때만 OOF로 집계 (일부 fold만으로 OOF라 부르지 않기)
  - 제출 스크립트가 학습 당시 설정 기록(`args.yaml`)으로 가중치를 고르고, 모호하면 멈추게 함
  - `assert` 대신 명시적 오류 (최적화 모드에서 검사가 꺼지는 문제)
  - README의 "단일 YOLO가 상한" 표현 → "fold 분할이 만든 상한일 뿐"으로 정정

추가로 한 일 – 5-Fold 교차검증과 최종 모델

- **YOLO11s·11m 5-Fold**: fold 평균 0.853 / 0.855로 차이 없음. fold별 편차가 0.74~0.92로 크다.
- **OOF가 0.62로 낮게 나온 원인 분석**: 56종 중 19종이 한 fold에만 있어, 검증될 때마다 학습에 없는 약이 된다 → OOF 상한 0.661.

fold별 "처음 보는 약" 수(1·3·3·5·7종)와 점수 순위가 정확히 반대라는 것도 확인했다.

→ 모델 비교는 같은 조건으로 잴 수 있는 **37종 OOF**로 하기로 함: 11s 0.941, 11m 0.936.

(희귀 19종을 뺀 조건부 비교값이라 56종 Kaggle 점수의 추정치는 아니다)

- **앙상블 설정을 OOF에서 선택**: 11s + 11m WBF가 0.954(+0.013), 1:1 · IoU 0.7.

다만 같은 OOF에서 12개 중 최댓값을 고른 값이라 낙관적일 수 있다 → 다음 날 근사 메타 검증으로 확인 (10/28 일지).

- **최종 모델**: 232장 전체로 11s 50 에포크 · 11m 83 에포크 학습 (fold별 best epoch 중앙값을 쓴 휴리스틱, 검증이 없으므로 마지막 가중치 사용).
- **제출 후보 3종 생성**: 11s+11m 앙상블(주력) · 11s 단일 · 11m 단일. 테스트 842장 모두 예측, 이미지당 확신 박스 3.6~3.7개(학습 데이터 3~4개와 일치).
- 최종 모델 폴더에 Ultralytics가 그린 PR 곡선·혼동 행렬은 **학습에 포함된 이미지로 계산된 것**이라 지웠다. 성능처럼 보이면 오해를 부른다.

고민 / 결정

- 자동화한 규칙의 결과는 반드시 눈으로 검수한다. 라벨 수정 규칙의 오탐 5건을 잡아낸 것처럼, 자동 처리 결과를 의심하고 검증하는 과정을 반드시 거친다.
- 학습 환경은 처음에 Kaggle 노트북으로 정했지만, 실험 20여 개를 빠르게 반복하려고 **로컬 GPU(RTX 3060)로 바꿨다** (Kaggle은 주간 GPU 시간 한도가 있다). 노트북은 결과 분석·시각화용으로 쓴다.
- GPU 하나에 학습 두 개를 동시에 돌리면 오히려 크게 느려졌다 → 학습은 대기열로 하나씩 순서대로 돌린다.
- 검증 점수는 **분할 구조**에 크게 좌우된다. 숫자 하나를 보기 전에 "이 숫자의 상한이 얼마인가"부터 계산한다.

못 한 일 / 내일로

- Kaggle 대회 페이지에서 평가 지표·제출 형식 확인 → 제출 후보 3종 제출, 점수 비교
- TTA는 OOF로 검증하지 않아 아직 끔 → 5-Fold 예측으로 효과 확인

- Copy-Paste 비율을 낮춘 재실험, 분석 노트북(04) 작성

소감

- 점수를 올리는 것보다 **점수가 왜 그렇게 나오는지 설명하는 것**이 더 어려웠고 더 많이 배웠다. 0.62라는 낮은 OOF가 모델 문제가 아니라 데이터 분할 구조 때문이라는 것을 숫자로 증명한 과정이 오늘의 가장 큰 수확이다.

업무일지 – 2026-10-28(수) · Day 2

오늘 할 일 (시작 전)

- 어제 결과(5-Fold, 최종 모델, 제출 후보)에 대한 외부 리뷰 지적 반영
- Kaggle 대회 페이지에서 제출 형식 확인 → 제출 후보 3종 제출
- 전처리 전후 비교(CLAHE) 실험 준비

한 일

- **결과 최종본 폴더 분리**: GitHub와 별도로 `pill-detection_최종본/`에 제출 CSV·모델 가중치·실험 결과·업무일지를 모으고, 목록 파일(기준 커밋·상태·MD5)을 만들었다. 제출 CSV와 가중치는 용량 때문에 Git에 올리지 않으므로 여기가 유일한 최종본이다.
- **클래스 이름 빈칸 수정**: 원본 데이터에 영문명이 없는 약(32310 글리아타민연질캡슐)이 있어 학습 그래프에 이름 없는 라벨이 찍혔다. 영문명이 없으면 약 코드(`id_32310`)로 대신한다.

리뷰 지적 반영

지적	확인 결과	조치
WBF 설정을 같은 OOF에서 12개 중 골라 최댓값(0.954)을 보고 → 낙관 편향	타당하다. 고른 데이터로 평가하면 선택 편향이 생긴다	**근사 메타 검증**: fold 하나를 보류하고 나머지 4개 fold로 설정을 고른 뒤 보류 fold에만 적용, 5번 반복. **5번 모두 같은 설정(1:1, IoU 0.7)이 뽑혀 설정 선택이 안정적** (보류 예측 합산 0.954)
"37종 OOF가 최종 모델에 더 가까운 추정치"라는 표현	과장이다. 희귀 19종을 정답·예측에서 모두 뺐으므로 어려운 클래스를 제외한 값이다	"모델·앙상블을 같은 조건에서 비교하는 **조건부 진단값**, 56종 Kaggle 점수 추정치가 아님"으로 README·계획서·업무일지 정정
"12개 설정 모두 높으므로 우연이 아니다"	12개 설정은 서로 상관돼 있어 독립 반복이 아니다	문장을 지우고 근사 메타 검증 결과로 대체
best epoch 중앙값이 최적이라는 근거 부족	타당하다. 데이터가 25% 늘고 학습률 스케줄이 총 에포크에 묶인다	"휴리스틱"으로 명시하고 **에포크 민감도** 분석 추가: P25·중앙값·P75 모두 최고점 대비 평균 손해 약 0.01 → 최고점 주변이 평평해 선택이 결과를 크게 바꾸지 않는다
OOF 집계가 파일 존재만 확인	맞다. 행 누락·중복·다른 fold 이미지가 섞여도 계산이 진행됐다	232장 완전 커버, fold 밖 이미지·예측 없는 이미지·중복 행이면 멈추도록 검사 추가 + 테스트
학습을 건너뛰고 평가할 때 설정 변경을 못 잡음	맞다. 설정 파일이 바뀌면 다른 실험의 결과로 기록될 수 있었다	학습 당시 기록(`args.yaml`)과 대조해 다르면 멈춤. Ultralytics 기본값까지 비교해, 설정 파일에 없는 증강 차이도 잡는다. **기존 학습 20개 전부 현재 설정과 일치** 확인

- 리뷰와 별개로 찾은 것:
  - 설정 파일 검사에 남아 있던 `assert`를 명시적 오류로 바꿨다 (최적화 모드에서 검사가 꺼지는 문제).
  - 전체 데이터 학습 때 학습 이미지로 계산된 검증 그림을 애초에 복사하지 않게 했다 (어제는 사후에 지웠다).
  - 요구사항 추적표가 실제 진행보다 뒤처져 있어 상태를 맞췄다 (R7·R8·R9·R10 완료).

색 보정 전처리 CLAHE 전후 비교 (R6)

- **가설**: Test가 더 어둡고 대비가 크다(EDA) → 이미지마다 국소 대비를 평탄화하면 분포 차이가 줄고 점수가 오를 것이다.
- **설계**: YOLO11s 5-Fold에서 전처리 하나만 바꿨다. 알약 색이 단서라 LAB 밝기 채널에만 적용했고, 설정값은 검증 점수로 고르지 않고 OpenCV 기본값으로 고정했다. 학습·검증·테스트가 같은 함수를 거치도록, 데이터셋 이름(`data_fold0_clahe.yaml`)만 보고 예측·제출 때 전처리를 자동으로 다시 적용하게 했다.
- **분포**: 측정 코드가 EDA 수치(밝기 131.5 / 125.9)를 그대로 재현하는지 먼저 확인했다. Test−Train 밝기 차이 −5.55 → −3.25, 대비 차이 +2.88 → +2.98 (그대로).
- **점수**: fold별 2개 상승·3개 하락, fold 평균 0.8530 → 0.8515. 37종 OOF는 0.941 → 0.951이지만 조합 단위 부트스트랩 95% 구간이 −0.006 ~ +0.027로 0을 포함한다.
- **결정**: 채택하지 않는다. 원본 + 학습 중 HSV 증강을 유지한다.

발견한 문제 (인사이트)

- **"고른 데이터로 평가하면 점수가 부풀려진다"는 원칙은 하이퍼파라미터뿐 아니라 앙상블 설정에도 적용된다.** 이번에는 근사 메타 검증에서도 같은 설정이 뽑혀 선택이 안정적이었지만, 그것은 확인한 뒤에야 말할 수 있다.
- **"보류했다"고 해서 격리된 것은 아니다.** 설정을 고를 때 쓴 다른 fold의 모델이 보류 fold를 학습했으므로, 처음에 "비편향 추정"이라고 쓴 것을 재검토에서 "근사 메타 검증(설정 안정성 점검)"으로 정정했다. 엄밀하게 하려면 보류 fold를 뺀 데이터 안에서 모델부터 다시 학습해야 한다.
- 자동화 코드는 "틀린 입력이면 멈추는지"까지 테스트해야 한다. 조용히 계산되는 틀린 숫자가 가장 위험하다.
- **합친 OOF가 오르는데 fold별로는 엇갈릴 수 있다.** 5개 모델의 예측을 한 순위로 섞어 평가하면 모델 간 점수 분포 차이도 결과에 섞인다. 그래서 점수 하나가 아니라 fold별 방향과 신뢰구간을 함께 보고 판단했다.
- 같은 조합을 여러 각도로 찍은 이미지는 독립이 아니다. 부트스트랩도 분할과 같은 단위(조합)로 뽑아야 구간이 지나치게 좁아지지 않는다.

고민 / 결정

- 에포크를 P75로 바꿀지 고민했다. P75의 손해가 가장 작아 보였지만, 조기 종료로 그 에포크까지 가지 못한 fold가 빠져 유리하게 치우친 값이었다. 빠진 fold도 넣어 보면 후보 간 차이가 0.01 안쪽이라 **재학습 없이 중앙값을 유지**했다.

못 한 일 / 내일로

- Kaggle 제출 형식 확인과 제출 (브라우저 연결 문제로 대기)
- TTA 효과를 5-Fold 예측으로 확인

소감

-

업무일지 – 2026-10-29(목) · Day 3

오늘 할 일 (시작 전)

- 보고서 초안 리뷰 지적 반영 (조합당 촬영 장수, 안전 논리, "최종" 표현, 쪽 번호, 참고문헌)
- 결과 분석(보고서 8장): 현재 최종 후보의 OOF 예측으로 클래스별 AP · 실패 유형 · 실패 사례

한 일

보고서 초안 보완

지적	확인 결과	조치
"모든 조합을 세 각도로 촬영"	틀렸다. 114조합 중 3장 40 · 2장 38 · 1장 36	README·보고서를 "최대 세 각도, 조합당 1~3장"으로 정정
"모르는 약은 신뢰도가 낮으면 거부"	앞의 근거(처음 보는 약을 0.96으로 오답)와 모순	신뢰도 임계값만으로는 부족하다고 정정. 운영 임계값은 제시하지 않음
남은 실험이 있는데 "최종"	맞다	"현재 최종 후보"로 통일
쪽 번호·참고문헌 없음, 제목이 쪽 끝에 홀로 남음	맞다	쪽 번호·머리글, 참고문헌 12개(DOI·원문 링크), 제목-본문 붙이기

- 표지 작성자 기입.

결과 분석 (`scripts/error_analysis.py`)

- mAP 하나를 나눠 "어디서 점수를 잃는가"를 보는 스크립트를 만들었다: 클래스별 AP, IoU 임계값별 mAP, 실패 유형(정답·위치 오류·분류 오류·놓침), 혼동 쌍, 신뢰도로 처음 보는 약을 가를 수 있는지(AUROC, 조합 단위 부트스트랩 구간).
- 앙상블이 단일 모델보다 나은 이유를 IoU별로 나눠 보니, 차이의 약 절반이 가장 엄격한 IoU 0.95 구간에서 나왔다 (원인별 가법 분해는 아님).
- 처음 보는 약의 4분의 1은 신뢰도 0.9 이상으로 다른 약 이름을 받았다 → "신뢰도 임계값만으로는 부족하다"의 정량 근거.

리뷰 지적 반영 (오류 분석)

- **실패 유형을 1:1 매칭으로 다시 짰다.** 처음 버전은 정답 박스마다 예측을 따로 찾아, 예측 하나가 겹친 두 정답에 동시에 쓰일 수 있었다. 점수 순으로 예측 하나를 정답 하나에만 배정하도록 바꾸고 회귀 테스트를 추가했다.
- **배경 오탐을 "학습 가능 37종" 그룹에 넣던 것을 별도 그룹으로 뺐다.** "보수적 처리"라는 이유로 집단 소속을 바꾸면 안 된다.
- 그림 가독성: 56종 세로 약명 그래프 → 하위 10종 가로 막대(전체는 부록 표), 실패 사례는 번호 + 표.

발견한 문제 (인사이트)

- **"점수 계산 규칙"도 검증 대상이다.** 실패 유형을 세는 규칙이 1:1이 아니면 숫자가 낙관적으로 나온다. 테스트를 "정상 입력"이 아니라 "겹친 두 정답 위 예측 하나" 같은 경계 사례로 만들어야 한다.

- **1:1로 바꾸자 달라진 4건을 끝까지 따라간 것이 다음 날의 큰 발견(라벨 오류)으로 이어졌다.** 숫자가 왜 바뀌었는지 설명할 수 없으면 그냥 넘어가지 않는다.
- 고정 신뢰도 임계값의 실패 유형 집계는 COCO mAP의 손실 분해가 아니다. "함께 관찰됐다"와 "원인이다"를 구분해 쓴다.

고민 / 결정

- 분석 대상은 현재 최종 후보(11s + 11m WBF)로 하고, 단일 모델은 핵심 수치만 비교 표로 둔다. 8장 전체를 모델마다 반복하면 보고서가 길어지기만 한다.
- 정확한 임계값 숫자(처음 보는 약 90%를 거르는 점)는 같은 OOF에서 고른 값이라 운영값이 아니라고 명시하고, 결론은 AUROC와 25.4%로 낸다.

못 한 일 / 내일로

- 1:1 매칭으로 바뀐 사례에서 원본 라벨 이상이 보여 원인 확인
- 학습률(R11)·TTA(R14) 실험

소감

-

업무일지 – 2026-10-30(금) · Day 4

오늘 할 일 (시작 전)

- 어제 1:1 매칭으로 바뀐 사례에서 보인 원본 라벨 이상 확인
- 학습률(R11)·TTA(R14) 실험
- 데모 앱, 보고서 완성 (Kaggle 제출·GitHub 반영은 다음 주)

한 일

원본 라벨 오류 4장 발견·수정

- 1:1 매칭으로 달라진 4건과 "학습한 약을 신뢰도 0.96~0.98로 틀렸다"는 사례를 원본 사진과 대조했다. 모델이 맞고 **라벨이 틀린** 경우였다.
  - 3장: 다른 약 두 개가 **같은 좌표**에 라벨되고, 알약 하나는 라벨이 없었다.
  - 1장: 두 알약의 약 코드가 **서로 바뀌어** 있었다.
- 같은 조합의 다른 각도 사진과 생김새를 비교해 확정했다. 남은 학습 가능 오분류·놓침 21건은 라벨이 맞았다.
- 정제 코드에 감사 두 개를 추가했다.
  - D: 다른 약 박스끼리 IoU ≥ 0.9이면 후보로 표시한다. 데이터만 보는 감사다.
  - E: OOF 예측이 정답과 신뢰도 0.9 이상으로 다르게 답하면 후보로 표시한다. 모델 예측을 쓰는 사후 감사다.
- 후보는 자동으로 고치지 않는다. 사람이 확인한 결정(`configs/label_review.csv`)만 적용한다.
- 무결성 검사에 "다른 약끼리 같은 좌표 0"을 추가했다.
- 리뷰 지적을 받아 결정 기록 형식을 바꿨다. 결정마다 ID, 검수자, 검수일, 검수 때 본 두 박스의 좌표를 적는다.
  - 현재 라벨이 그 좌표와 같을 때만 적용하므로, 다시 적용해도 라벨 교환이 원상복귀하지 않는다.
  - 실제로 겹친 알약은 keep_overlap으로 남길 수 있다.
- 형식 검사와 경계 사례(미확정 결정, 좌표 불일치, 박스·알약 수 불일치)를 테스트했다.
- 재검토 지적을 받아 두 가지를 더 고쳤다.
  - keep_overlap도 검수 때 본 좌표 그대로일 때만 겹침을 허용한다. 두 박스가 함께 다른 곳으로 옮겨진 겹침은 새 후보로 잡힌다.
  - 감사 후보에는 정확히 같은 두 약 쌍의 결정만 붙인다. 같은 박스를 고친 다른 쌍의 결정은 "관련 결정"으로 구분해 적는다.
  - 데이터 준비를 다시 돌려 정제 결과와 fold가 그대로임을 확인했다.
- 3차 재검토 지적을 반영했다.
  - 감사 스크립트가 Windows 기본 콘솔(cp949)에서 대시 문자 때문에 멈췄다. 콘솔에 찍는 문자열의 cp949 밖 문자(대시, 경고 기호)를 모두 바꾸고, 이를 검사하는 테스트를 추가했다. 인코딩 설정 없이 실행해 정상 종료를 확인했다.
  - 결정이 없는 후보를 "기각"으로 표시하던 기본값을 "미검토"로 바꾸고, 미검토 후보가 있으면 감사 스크립트가 실패로 끝나게 했다. 라벨이 맞다는 기각은 keep_label 결정으로만 남긴다.
- 4차 재검토 지적을 반영했다.

    - 감사 출처를 예측 파일 해시 하나에서 예측 · 입력(원본 라벨 · 이미지 · fold · 검수 결정) · 코드 세 묶음의 manifest 해시로 넓혔다.
    - 검수 결정 파일에 의미 검사(규칙별 허용 결정, 같은 약 코드, 빈 검수 정보, 날짜 형식, 좌표 범위)를 추가했다.
    - 스크립트 설명(--help 출력)의 대시도 기본 콘솔에서 멈추지 않게 바꿨다.
    - 테스트 기록(reports/logs/pytest.log)을 다시 만들었다. 테스트 함수 35개, 매개변수 포함 37개 케이스가 통과했다.
- 감사 결과에 출처(실험 이름, WBF 가중치·IoU, 예측 파일 해시)를 남겼다. 수정 전 1:1 감사는 E 4건, 수정 후 2:1 재감사는 E 5건이고, 늘어난 1건은 이미 고친 박스였다.
- 박스 수는 771개(원본 763 + 누락 추가 8) 그대로이고, 수정 기록은 10건 → 15건(이동 4 · 교환 2 포함)이 되었다. 분할(`folds.csv`)은 바뀌지 않았다.

전체 재학습 (5-Fold 15회 + 최종 모델 2개)

- 라벨이 바뀌었으므로 11s·11m·CLAHE 5-Fold와 최종 모델을 모두 다시 학습했다. 이전 결과는 `runs/_before_label_fix/`에 보관했다.

37종 조건부 OOF mAP50-95	수정 전	수정 후
YOLO11s	0.941	0.956
YOLO11m	0.936	0.941
WBF 11s + 11m	0.954 (1:1)	0.964 (2:1)

- 수정 전후 차이를 "정제 효과"로 쓰지 않는다. 평가 정답도 함께 바뀌었으므로, 수정된 라벨에서 새로 잰 추정치로만 보고한다.
- **WBF 설정이 1:1 → 2:1로 바뀌었다.** 같은 절차(근사 메타 검증)를 다시 돌리자 보류 fold 5개 모두에서 2:1이 뽑혔다. 절차를 고정하고 결과를 따랐다. IoU는 3개 fold에서 0.7, 2개 fold에서 0.6이 뽑혔다.
- 최종 모델 에포크도 같은 규칙(fold별 best epoch 중앙값)으로 다시 계산해 11s 50 → 80, 11m 83 → 80으로 바꿨다.

학습률 실험 (R11)

- optimizer=auto가 실제로 고른 값은 AdamW, lr0 0.000167이다. 이것을 기준으로 3배·0.3배만 바꿔 fold 0에서 비교했다.
- 판단 규칙은 실행 전에 설정 파일에 적어 두었다: "E0보다 0.005 이상 좋을 때만 5-Fold로 확인한다."

학습률	fold 0 mAP50-95	E0 대비
0.000167 (E0)	0.8356	–
0.0005 (3배)	0.8381	+0.0025
0.00005 (0.3배)	0.8269	−0.0087

- 3배는 기준에 못 미치고, 0.3배는 떨어졌다. 그래서 auto 값을 유지한다.

TTA (R14)

- 학습 없이 5-Fold 모델로 TTA 검증 예측을 다시 만들어(`scripts/predict_oof.py`) 기준 예측과 짝지어 비교했다.

- 37종 OOF가 11s −0.004(조합 단위 부트스트랩 95% 구간 −0.011 ~ +0.002), 11m −0.005(−0.010 ~ +0.002)로 둘 다 내려갔다. 개선 확률은 9%, 11%다.
- 그래서 미채택한다. 고정 카메라 데이터라 축소 변환이 학습 분포 밖의 크기를 만들었을 수 있다 (가설).

CLAHE 재검증

- 수정 라벨로 다시 학습해 비교했다. 2개 fold 상승 · 3개 하락(처음 보는 약이 가장 많은 fold 3에서 −0.015)이고, 37종 OOF는 +0.005(조합 단위 부트스트랩 95% 구간 −0.003 ~ +0.018)였다.
- 라벨 수정 전과 같은 결론이라 계속 미채택한다.

데모 앱 (R16)

- `streamlit run app/demo.py`로 실행한다. 사진을 올리면 알약 위치, 약 이름, 신뢰도, 다른 후보를 보여 준다.
- 신뢰도가 0.5 미만이거나 같은 알약에 이름이 둘 이상 붙으면 "확인 필요"로 표시한다. 이 표시도 오답을 모두 거르지는 못한다는 경고를 함께 띄운다.
- gradio를 설치하려다 공용 환경의 패키지 버전이 충돌해 되돌렸다. 데모는 이미 설치된 Streamlit으로 만들었다.

보고서·README

- 3장(감사 D·E), 6.7(학습률·TTA), 10장(타임라인·검토 반영 이력·발표 참여), 12장(회고)을 쓰고, 모든 수치를 수정 라벨 기준으로 바꿨다.
- 8장은 "학습한 약끼리 고신뢰 오분류" 결론을 철회했다. 그 사례 대부분이 라벨 오류였기 때문이다.

발견한 문제 (인사이트)

- **검사 항목을 "가능한 오류 유형"에서 출발해 정해야 한다.** 처음 무결성 검사는 "같은 약 중복"만 보고 "다른 약끼리 같은 좌표"를 놓쳤다.
- **모델 예측으로 찾은 라벨 오류 후보는 모델이 같이 틀린 오류를 놓친다.** 그래서 탐지 재현율을 알 수 없다고 명시했다. 데이터만 보는 D와 구분해 기록했다.
- **선택 절차를 고정하면 결과가 바뀌어도 흔들리지 않는다.** 라벨 수정 뒤 WBF 가중치와 에포크가 바뀌었지만, 같은 규칙을 다시 적용했을 뿐이라 설명이 쉽다.
- 백그라운드 대기열을 중지했다고 생각했는데 프로세스가 남아, 같은 학습이 두 번 동시에 돌았다. 프로세스 목록으로 확인한 뒤 둘 다 끄고 하나만 다시 돌렸다. 중지 후에는 반드시 프로세스 목록으로 확인한다.

고민 / 결정

- 라벨 수정 후 재학습 범위: fold 0만 다시 돌리면 5-Fold 결론과 섞인다. 그래서 5-Fold 전체와 최종 모델을 다시 학습했다.
- 라벨 수정 전 fold 0 탐색(E1~E3, 2단계, 분류기, Copy-Paste)은 다시 돌리지 않았다. "라벨 수정 전 기록"으로 표시하고, 최종 수치와 직접 비교하지 않는다.

못 한 일 / 다음 주

- Kaggle 제출 형식 확인과 제출, 점수 기입 (보고서 1.3·7.2)

- GitHub 반영, 시연 영상 녹화 (11/03)

소감

-

업무일지 – 2026-11-02(월) · Day 5

오늘 할 일 (시작 전)

- 재검토 지적 반영, 보고서 쪽 나눔 정리, 결과 최종본 폴더 갱신

한 일

- `b4c319f`: 37종 OOF 설명을 "조건부 진단값(56종 · Kaggle 성능 추정치 아님)"으로, 감사 해시 재현 조건을 "동일 의존성 환경에서"로 한정했다.
- `16ca344`: 보고서를 23쪽에서 22쪽으로 정리했다.
  - 6.7절의 제목과 표를 같은 쪽에 묶고, 7 · 8 · 9장과 부록 B를 앞 내용에 이어 붙였다.
  - 부록 A를 37종 · 19종 표로 나누고 글자를 키웠다.
  - 그림 8-1 제목의 "19종 모두 AP 0"을 동아가바펜틴정(0.135)에 맞게 고치고, 범례 겹침을 없앴다. 오류 분석을 다시 돌려 CSV가 그대로임을 확인했다.
- `8b99672`: `requirements.txt`의 pandas · Pillow를 최종 모델을 만든 환경 버전(2.3.3, 11.3.0)에 맞췄다.
- 결과 최종본 폴더를 라벨 수정 후 기준으로 갱신하고, 이전 파일은 별도 폴더로 옮겼다.

발견한 문제 (인사이트)

- `requirements.txt`와 실제 설치 버전이 달랐다. 설치 폴더 시각이 10/30 09:34라, 재학습 도중 환경이 바뀌었을 수 있다. 처음에는 streamlit 설치 때문이라고 짐작했지만 요구 조건(pandas<4, pillow<13)을 보니 근거가 없어 원인 불명으로 남겼다.
- 앞으로는 실험 기록에 라이브러리 버전을 함께 남겨야 시각으로 추정하지 않아도 된다.

고민 / 결정

- 쪽 나눔은 Kaggle 점수 기입 뒤에 다시 확인하기로 했다.

이력 기록

- `b4c319f`는 최종 판정(모든 검토 통과) 전에 원격 `main`에 올라갔다. "최종 판정 후 push" 규칙과 어긋나므로 사실대로 남긴다. 이후 커밋은 로컬에 두었다.

못 한 일 / 다음

- Kaggle 제출, 시연 영상, 최종 push

소감

-

업무일지 – 2026-11-03(화) · Day 6

오늘 할 일 (시작 전)

- 기록 없음

한 일

- 저장소에 이 날의 커밋, 파일 변경, 학습 기록이 없다.

발견한 문제 (인사이트)

- 없음

고민 / 결정

- 없음

못 한 일 / 다음

- 시연 영상, Kaggle 제출, 최종 반영은 11/05로 넘어갔다.

소감

-

업무일지 – 2026-11-04(수) · Day 7

오늘 할 일 (시작 전)

- 기록 없음

한 일

- 저장소에 이 날의 커밋, 파일 변경, 학습 기록이 없다.

발견한 문제 (인사이트)

- 없음

고민 / 결정

- 없음

못 한 일 / 다음

- 시연 영상, Kaggle 제출, 최종 반영은 11/05로 넘어갔다.

소감

-

업무일지 – 2026-11-05(목) · Day 8

오늘 할 일 (시작 전)

- 저장소 · 보고서 마감(19:00)까지 Kaggle 제출, 점수 기입, 시연 영상, 최종 반영
- 재검토에서 남은 재현성 항목 정리

한 일

- 공지를 다시 읽고 일정을 바로잡았다. 처음 적어 둔 마감(11/04, 11/05)이 틀렸고, 실제로는 저장소 · 보고서 11/05 19:00, 업무일지 · Kaggle 11/06 23:50이다.
- 재현 조건을 문서에 공개했다.
  - 일부 5-Fold 산출물이 다른 pandas · Pillow 버전에서 만들어졌을 수 있다는 점을 파일 시각으로 범위를 좁혀 적었다.
  - 새 가상환경에 `requirements.txt`를 설치해 의존성 검사, 단위 테스트, 라벨 감사 재현을 확인했다. 다른 PC와 GPU 학습 재현은 확인하지 못했다.
- GitHub 1차 push를 했다 (`dff688b`). 원격과 로컬 커밋이 같은 것을 확인했다. 최종 판정 전이지만 직접 판단해 올렸다.
- 대회 페이지를 처음으로 자세히 읽고 평가 지표가 **mAP@[0.75:0.95]** 임을 확인했다.
  - 페이지에는 표기만 있어 COCO 관례(0.05 간격 5개)로 해석한 내부 대리지표로 계산한다. 주최 측 평가와 같은지는 확인하지 못했다. 정의는 `configs/evaluation.yaml`에 두고 결과 CSV에도 함께 남긴다.
  - `coco_map`에 이 지표를 추가하고 테스트를 넣었다 (IoU 0.8 예측이면 5개 기준 중 2개만 맞아 0.4).
  - 학습 없이 저장된 OOF로 최종 결정을 다시 평가했다. 가중치 2:1, TTA · CLAHE 미채택은 그대로이고, WBF IoU만 0.7에서 0.6으로 바뀌었다.
  - 주력 제출 파일 `final_11s_11m_wbf_iou06`을 새로 만들고, 8장 오류 분석과 부록 A를 이 설정으로 다시 계산했다.
- 재검토에서 지목받은 항목을 고쳤다.
  - 지표 정의를 `configs/evaluation.yaml`로 옮겼는데 maxDets · 보간 · 평균 방식은 기록만 되고 계산에 쓰이지 않았다. maxDets를 실제 계산에 적용하고, 지원하지 않는 값과 격자에 없는 IoU는 오류로 멈추게 했다. 테스트는 함수 37개, 케이스 39개다.
  - CLAHE · TTA 비교 CSV에 대리지표의 합산 OOF 행과 지표 정의를 넣어 다시 만들었다.
- 데모 앱을 실제로 실행해 확인하고 시연 영상(`docs/demo/demo.mp4`, 45초)을 만들었다. 실행 화면을 캡처해 자막과 엮었고, 음성은 없다.
  - 신뢰도 0.5 미만 알약에 "확인 필요"가 표시되는 것(테스트 88.png)을 영상에 넣었다.
- 메타 검증에서 fold 2에 1:2가 뽑히는 현상을 보고 원인을 찾았다. 소수 넷째 자리로 반올림한 값으로 비교해 생긴 가짜 동점이었고, 반올림 전 값으로 고르도록 고쳤다.
- Kaggle에 후보 4개를 제출했다. 주력 WBF(2:1, IoU 0.6) 0.41349, WBF(IoU 0.7) 0.41410, YOLO11s 0.40954, YOLO11m 0.39427. 모두 정상 채점되어 제출 형식과 약 코드 체계를 확인했다.
- 점수를 보고서 표지 · 7.2절, README, PLAN에 넣고 원인 가설을 정리했다. 데이터 설명에서 금지는 두 파일뿐임을 다시 읽고 보고서의 제약 설명도 고쳤다.
- 리더보드를 보니 상위 4팀이 0.611~0.634로 격차가 컸다. 데이터 설명을 다시 읽고 허용된 AI Hub 조합 데이터로 후속 실험을 시작했다.

    - 금지 묶음을 뺀 라벨 7묶음 분석: 약 116종, 대회 56종 중 54종 겹침, 새 약 62종.
    - 내려받기 때 금지 파일(`TL_2_조합.zip`, `TS_2_조합.zip`)이 함께 받아졌다. 열지 않았고, 변환 코드가 이 이름을 건너뛰게 했다.
    - 변환 스크립트(`scripts/prepare_aihub.py`)를 만들었다. 라벨이 빠진 이미지, 테스트와 같은 사진이 있는 조합을 제외하고 제외 내역을 남긴다.
    - 1묶음으로 15 에포크 시험 학습: 대회 fold 0 mAP50-95 0.835 → 0.937, Kaggle public 0.40954 → 0.52212 (YOLO11s 단일 기준).
    - 전체 7묶음(10,175장, 118클래스)으로 학습했다. 검증 최고 지점인 28 에포크로 전체 데이터를 학습해 제출하니 Kaggle public 0.63017이 나왔다.

발견한 문제 (인사이트)

- 대회 평가 지표를 프로젝트 초반에 확인하지 않은 것이 가장 큰 실수다. "제출 때 보면 된다"고 미뤘고, 그동안 모든 실험을 다른 지표로 비교했다. 다행히 순위는 그대로였지만 운이 좋았던 것이다. 다음 프로젝트에서는 첫날 평가 지표와 제출 형식부터 확인한다.
- 엄격한 구간만 보면 앙상블 이득이 커진다 (mAP50-95 +0.008 → 대리지표 +0.012). 앙상블 이득의 절반이 IoU 0.95에서 나온다는 앞선 분석과 맞는다.
- 처음 보는 약의 신뢰도 수치는 WBF 설정에 민감했다. mAP가 거의 같은 IoU 0.6과 0.7 사이에서 "0.9 이상 비율"이 24%와 30%, AUROC가 0.884와 0.842로 달랐다. 63박스 표본의 숫자를 정밀한 값으로 읽으면 안 된다.
- Kaggle 점수가 내부 대리지표보다 크게 낮았다 (0.413 vs 37종 0.950, 56종 0.628). 후보 순위는 OOF와 같았지만 절댓값은 맞지 않았다. 대회 안내대로 테스트에 학습에 없는 클래스가 있는 것이 유력한 이유인데, 신뢰도 분포로 추정한 해당 박스 비율(약 9~11%)만 보면 전부 설명된다고 단정할 수 없다. 정답이 비공개라 확정은 못 한다.
- 데이터 설명을 끝까지 읽지 않았던 것도 지표를 늦게 본 실수와 같은 종류다. 허용된 추가 AI Hub 데이터를 제출 전에 검토하지 않았다.

고민 / 결정

- 0.6과 0.7의 차이는 0.0007로 작다. 그래도 미리 정한 규칙(보류 선택에서 더 자주 뽑힌 값)을 그대로 따랐다. Kaggle Public 점수를 보고 후보를 다시 고르지 않기로 했다.

못 한 일 / 다음

- 전체 데이터 학습 결과 확인과 Kaggle 제출
- LMS 제출

소감

-